```
# ==================== GENERATIVE AI LAB 9 ====================
# Sequence Generation using LSTM and Transformer

import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Embedding, LSTM, Dense, Input, LayerNormalization, MultiHeadAttention, Dropout
from tensorflow.keras.utils import to_categorical

# ---------------------- DATASET ----------------------
data = """
machine learning models learn patterns from data.
sequence models process data step by step.
recurrent neural networks are designed for sequential tasks.
rnn models maintain hidden states across time steps.

long short term memory networks solve long dependency problems.
lstm uses gates to control information flow.
gru models simplify the lstm architecture.
sequence prediction is useful in many applications.

language modeling predicts the next word in a sentence.
speech recognition processes audio sequences.
time series forecasting predicts future values.
music generation creates new melodies.

generative models learn probability distributions.
they generate new samples similar to training data.
sequence generation is widely used in artificial intelligence.
deep learning improves sequence modeling performance.
"""

sentences = [line.strip() for line in data.split("\n") if line.strip()]

# ---------------------- TOKENIZATION ----------------------
tokenizer = Tokenizer()
tokenizer.fit_on_texts(sentences)
total_words = len(tokenizer.word_index) + 1

# ---------------------- SEQUENCE CREATION ----------------------
input_sequences = []
for line in sentences:
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        input_sequences.append(token_list[:i+1])

max_len = max(len(seq) for seq in input_sequences)

input_sequences = pad_sequences(input_sequences, maxlen=max_len, padding='pre')
X = input_sequences[:, :-1]
y = input_sequences[:, -1]
y = to_categorical(y, num_classes=total_words)

# ==================== COMPONENT I: LSTM ====================
print("Training LSTM Model...")

lstm_model = Sequential([
    Embedding(total_words, 64, input_length=max_len-1),
    LSTM(100),
    Dense(total_words, activation='softmax')
])

lstm_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
lstm_model.fit(X, y, epochs=80, verbose=0)

def generate_lstm(seed_text, next_words=8):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=max_len-1, padding='pre')
        predicted = np.argmax(lstm_model.predict(token_list, verbose=0))

        for word, index in tokenizer.word_index.items():
            if index == predicted:
                seed_text += " " + word
                break
    return seed_text

print("\nLSTM Output:")
print(generate_lstm("machine learning"))

# ==================== COMPONENT II: TRANSFORMER ====================
print("\nTraining Transformer Model...")

# Positional Encoding
def positional_encoding(max_len, d_model):
    pos = np.arange(max_len)[:, np.newaxis]
    i = np.arange(d_model)[np.newaxis, :]
    angle_rates = 1 / np.power(10000, (2 * (i//2)) / np.float32(d_model))
    angle_rads = pos * angle_rates

    angle_rads[:, 0::2] = np.sin(angle_rads[:, 0::2])
    angle_rads[:, 1::2] = np.cos(angle_rads[:, 1::2])

    return tf.cast(angle_rads[np.newaxis, ...], dtype=tf.float32)

class TransformerBlock(tf.keras.layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim):
        super().__init__()
        self.att = MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
        self.ffn = Sequential([
            Dense(ff_dim, activation="relu"),
            Dense(embed_dim),
        ])
        self.norm1 = LayerNormalization()
```

```
        self.norm2 = LayerNormalization()
        self.dropout1 = Dropout(0.1)
        self.dropout2 = Dropout(0.1)

    def call(self, inputs):
        attn_output = self.att(inputs, inputs)
        out1 = self.norm1(inputs + self.dropout1(attn_output))
        ffn_output = self.ffn(out1)
        return self.norm2(out1 + self.dropout2(ffn_output))

embed_dim = 64
num_heads = 2
ff_dim = 64

inputs = Input(shape=(max_len-1,))
embedding = Embedding(total_words, embed_dim)(inputs)

pos_encoding = positional_encoding(max_len-1, embed_dim)
x = embedding + pos_encoding

x = TransformerBlock(embed_dim, num_heads, ff_dim)(x)
x = tf.keras.layers.GlobalAveragePooling1D()(x)

outputs = Dense(total_words, activation="softmax")(x)

transformer_model = Model(inputs, outputs)
transformer_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
transformer_model.fit(X, y, epochs=80, verbose=0)

def generate_transformer(seed_text, next_words=8):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=max_len-1, padding='pre')
        predicted = np.argmax(transformer_model.predict(token_list, verbose=0))

        for word, index in tokenizer.word_index.items():
            if index == predicted:
                seed_text += " " + word
                break
    return seed_text

print("\nTransformer Output:")
print(generate_transformer("deep learning"))

# ==================== END ====================
```

```
Training LSTM Model...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(

LSTM Output:
machine learning models learn patterns from data data step steps

Training Transformer Model...

Transformer Output:
deep learning improves sequence modeling performance next word in a
```